ESP32 PROGRAM USING ARDUINO IDE

Program code:

```
#include <ArduinoJson.h>
#include <WiFi.h>
#include <IOXhop_FirebaseESP32.h>

// Firebase and WiFi configuration
#define FIREBASE_HOST "https://xolcanodashboard.firebaseio.com/"
#define FIREBASE_AUTH "bhvXEUImDKwYu2ruN36Fpg3JBCSstKILK9qMXcxD"
#define WIFI_SSID     "XOLCANO"
#define WIFI_PASSWORD "goodluckckm18#1"

#define RELAY_PIN 18              // Relay connected to GPIO18
#define TIME_CHECK_INTERVAL 10000  // Check scheduled time every 10 seconds

// Initialize HardwareSerial on UART1 (adjust pins as needed)
HardwareSerial mySerial(1);

// Global flags for alarm triggers
bool relayTriggered1 = false; // For alarm from /FirebaseApp/value5
bool relayTriggered2 = false; // For alarm from /FirebaseApp/value6

unsigned long lastTimeCheck = 0;

//-----------------------------------------
// Utility: Sends a raw frame to the DWIN display over UART.
void sendDwinFrame(const uint8_t *data, size_t length) {
mySerial.write(data, length);
 mySerial.flush();
}

//-----------------------------------------
// Utility: Builds and sends a frame to update text at a given memory address.
void sendTextToDwinAtAddress(const char *text, uint16_t addr) {
  size_t textLength = strlen(text);
  size_t frameLength = 6 + textLength;
  uint8_t frame[128];
  int index = 0;

  frame[index++] = 0x5A;
```

```
  frame[index++] = 0xA5;
  frame[index++] = frameLength - 3;
  frame[index++] = 0x82;
  frame[index++] = (addr >> 8) & 0xFF;
  frame[index++] = addr & 0xFF;

  for (size_t i = 0; i < textLength; i++) {
    frame[index++] = text[i];
  }

  sendDwinFrame(frame, frameLength);
}

//-----------------------------------------
// Utility: Clears the DWIN display by sending blank strings to each
memory address.
void clearDwinScreen() {
  const char *blank = "
                        "; // 20 spaces
  sendTextToDwinAtAddress(blank, 0x3000);
  sendTextToDwinAtAddress(blank, 0x4000);
  sendTextToDwinAtAddress(blank, 0x6000);
  sendTextToDwinAtAddress(blank, 0x7000);
  delay(200);
}

//-----------------------------------------
// Predefined page commands for changing display pages.
uint8_t pages[][10] = {
  {0x5A, 0xA5, 0x07, 0x82, 0x00, 0x84, 0x5A, 0x01, 0x00, 0x00},
  {0x5A, 0xA5, 0x07, 0x82, 0x00, 0x84, 0x5A, 0x01, 0x00, 0x01},
  {0x5A, 0xA5, 0x07, 0x82, 0x00, 0x84, 0x5A, 0x01, 0x00, 0x02},
  {0x5A, 0xA5, 0x07, 0x82, 0x00, 0x84, 0x5A, 0x01, 0x00, 0x03},
  {0x5A, 0xA5, 0x07, 0x82, 0x00, 0x84, 0x5A, 0x01, 0x00, 0x04},
  {0x5A, 0xA5, 0x07, 0x82, 0x00, 0x84, 0x5A, 0x01, 0x00, 0x05},
  {0x5A, 0xA5, 0x07, 0x82, 0x00, 0x84, 0x5A, 0x01, 0x00, 0x06},
  {0x5A, 0xA5, 0x07, 0x82, 0x00, 0x84, 0x5A, 0x01, 0x00, 0x07}
};

const int numPages = sizeof(pages) / sizeof(pages[0]);
int currentPage = 0;
unsigned long lastPageChange = 0;

//-----------------------------------------
```

```
// Get current time from the network in HHMM format using NTP.
String getCurrentTime() {
 struct tm timeinfo;
 if (!getLocalTime(&timeinfo)) {
  Serial.println("Failed to obtain time");
  return "";
 }

 char timeStr[5]; // Format: "HHMM"
 snprintf(timeStr, sizeof(timeStr), "%02d%02d", timeinfo.tm_hour,
timeinfo.tm_min);
 return String(timeStr);
}

//-----------------------------------------
// Check scheduled times (from two Firebase nodes) and control relay.
void checkScheduledTime() {
  // Retrieve first scheduled time from Firebase (e.g., "1300" for 1:00 PM)
  String scheduledTime1 = Firebase.getString("/FirebaseApp/value5");
  scheduledTime1.replace("\"", "");
  scheduledTime1.replace("\\", "");

  // Retrieve second scheduled time from Firebase (e.g., "1405" for 2:05 PM)
  String scheduledTime2 = Firebase.getString("/FirebaseApp/value6");
  scheduledTime2.replace("\"", "");
  scheduledTime2.replace("\\", "");

  String currentTime = getCurrentTime();

  Serial.print("Current Time: ");
  Serial.print(currentTime);
  Serial.print(" | Alarm1: ");
  Serial.print(scheduledTime1);
  Serial.print(" | Alarm2: ");
  Serial.println(scheduledTime2);

  // Check Alarm 1 (Firebase value5)
  if (scheduledTime1.length() > 0 && currentTime == scheduledTime1 &&
!relayTriggered1) {
    Serial.println("Alarm 1 triggered! Turning on relay.");
    digitalWrite(RELAY_PIN, HIGH);
    delay(1500); // Relay ON for 10 seconds
    digitalWrite(RELAY_PIN, LOW);
    Serial.println("Alarm 1: Relay turned OFF.");
```

```
  relayTriggered1 = true;
 } else if (currentTime != scheduledTime1) {
  // Clear flag when time moves away so that next day's alarm can trigger.
  relayTriggered1 = false;
 }

 // Check Alarm 2 (Firebase value6)
 if (scheduledTime2.length() > 0 && currentTime == scheduledTime2 &&
!relayTriggered2) {
  Serial.println("Alarm 2 triggered! Turning on relay.");
  digitalWrite(RELAY_PIN, HIGH);
  delay(500); // Relay ON for 10 seconds
  digitalWrite(RELAY_PIN, LOW);
  Serial.println("Alarm 2: Relay turned OFF.");
  relayTriggered2 = true;
 } else if (currentTime != scheduledTime2) {
  relayTriggered2 = false;
 }
}

void setup() {
 Serial.begin(115200);
 mySerial.begin(115200, SERIAL_8N1, 17, 16);
 delay(1000);

 // Connect to WiFi.
 WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
 while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
 }
 Serial.println("\nWiFi Connected!");

 // Initialize Firebase.
 Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);

 // Get network time using NTP with an offset for +5:30 (19800 seconds).
 configTime(19800, 0, "pool.ntp.org", "time.nist.gov");

 // Set relay pin as output.
 pinMode(RELAY_PIN, OUTPUT);
 digitalWrite(RELAY_PIN, LOW);
}
```

```
void loop() {
 // Cycle display pages every 5 seconds.
 if (millis() - lastPageChange > 5000) {
  sendDwinFrame(pages[currentPage], sizeof(pages[0]));
  currentPage = (currentPage + 1) % numPages;
  lastPageChange = millis();
 }

 // Retrieve display values from Firebase.
 String value3000 = Firebase.getString("/FirebaseApp/value1");
 value3000.replace("\"", "");
 value3000.replace("\\", "");

 String value4000 = Firebase.getString("/FirebaseApp/value2");
 value4000.replace("\"", "");
 value4000.replace("\\", "");

 String value6000 = Firebase.getString("/FirebaseApp/value3");
 value6000.replace("\"", "");
 value6000.replace("\\", "");

 String value7000 = Firebase.getString("/FirebaseApp/value4");
 value7000.replace("\"", "");
 value7000.replace("\\", "");

 // Clear the display before updating.
 clearDwinScreen();

 // Update DWIN display memory addresses.
 sendTextToDwinAtAddress(value3000.c_str(), 0x3000);
 sendTextToDwinAtAddress(value4000.c_str(), 0x4000);
 sendTextToDwinAtAddress(value6000.c_str(), 0x6000);
 sendTextToDwinAtAddress(value7000.c_str(),  0x7000);

 // Check scheduled times and control relay every TIME_CHECK_INTERVAL.
 if (millis() - lastTimeCheck > TIME_CHECK_INTERVAL) {
 checkScheduledTime();
  lastTimeCheck = millis();
 }
```

CHAPTER 4:

4.1 RESULTS AND OBSERVATIONS

Performance Analysis

- The ESP32 maintains a stable connection with Firebase.
- The DWIN display updates instantly when a new message is received.
- Scheduled relay activation works accurately based on Firebase time values.
- The MIT App Inventor application provides a user-friendly interface for updates.

4.2 Challenges Faced:

- Initial synchronization of Firebase and ESP32 required debugging.
- Ensuring reliable communication between ESP32 and DWIN display.
- Managing power stability for uninterrupted operation.

4.3 Advantages

1. Real time updates:

- Notifications are updated instantly using the android application, reducing manual efforts.
- Students and faculty can receive timely notifications about schedules changes exam alerts, or urgent announcements.
- The ESP32's Wi-Fi capabilities enable instant updates to the notice board from a remote server or mobile app.

2. Wireless communication:

- Eliminates the need for physical connections between the notice board and the control system, making installation easier.
- Can be integrated with cloud-based services for centralized control.

4.4 Disadvantage:

- The ESP32, while powerful for its size, has limited processing and memory capacity.
- Limited GPIO and peripheral handling under load.
- Can lead to timing issues or lag.
- This can impact the accuracy of the exam bell timings.

4.5 Future scope:

The ESP32-based smart notice board with an integrated exam belling system holds vast potential for future enhancements. One of the key developments could be the integration of a mobile application, allowing remote scheduling of bells and uploading of notices via smartphones. Additionally, cloud connectivity can enable centralized management of multiple boards across different classrooms or campuses. Voice announcement features using text-to-speech engines can provide clearer alerts, while support for multi-language displays can cater to diverse user groups. Real-time network clock with battery backup could ensure accurate timing during power interruptions. Furthermore, a web-based dashboard and data logging features can help administrators monitor system performance and automate tasks. Future versions may also include biometric security, AI-based schedule prediction, and smart energy monitoring for better reliability, security, and efficiency.

4.5 Applications

1. Educational Institutions:

- Automates colleges and school announcements, exam schedules, and event notifications.
- Helps in classroom management with automated bell ringing.

2. Corporate Offices:

- Used in offices for displaying meetings schedules, employee notice, and company updates.
- Showcase training schedules, deadlines, corporative event, ensuring employees are well prepared.

3. Hospitals & Clinics:

- Displays patient appointment details, doctor availability, and emergency notifications.
- Alerts medical students and staff about exams, meetings, and emergency codes.
- Provides health awareness messages and updates on hospital policies.

4. Government & Public institutions:

- Displays public services updates, notices, and policies changes.
- In courts and municipal offices, it shows hearing schedules and appointments.
- Real time weather warnings, emergency alters, and public event notification.

5. Shopping malls & Transportation hubs:

- Provides advertisements, announcements, and store promotions.
- In bis stops, railway stations, and airports, it shows real time schedules and delays.
- In parking lots, it provides vacant space information and alerts.